

atomOS ASA



Operator engine addendum Log-polar LUT & texture kernel

Paired aperture. NA after ASA. Liquid-lens lookup.
Ommatidia image tiles. Explicit Morton swizzle.

Tom Klootwijk

Author identifier	NL200678942
Author date, as supplied	10-07-1990
Device profile	GeForce RTX 5070 Ti Laptop GPU
Nominal device memory	12 GB GDDR7
Native CUDA target	sm_120
Edition	3.0.0 / 13 September 2026

Verification-ready source edition

A standalone numerical image-operator module with shared C++/CUDA semantics, an independent Python oracle, explicit source-to-code assignments, runtime memory checks and a Codex repair brief.

The accompanying ZIP contains source, tests, executed CPU evidence and editable typesetting files. Hardware model and compiler target are documented in H1–H3. The implementation and its test status are specified separately.

The addendum at a glance

This edition formalizes the new **ASA / >0<** layer rather than repeating the entire preceding engine specification. Its numerical path is

$$\boxed{\text{ASA pair} \rightarrow \text{NA gate} \rightarrow \text{lens LUT} \rightarrow \text{log-polar key} \rightarrow \text{swizzle} \rightarrow \text{image result.}} \quad (1)$$

The source defines **>** and **<** as paired mirrors around the central 0 aperture, puts numerical aperture after ASA, and closes the addendum with a GPU address swizzle. These are the organizing elements of the implementation. [A, pp. 63–67, 75–87]

Source A is the 91-page *WHITE KING / ballistic trajectory cloud chamber* PDF. **Source B** is the eight-page *fly's-have-interesting-eyes-how-do-they-work* PDF. The latter contributes the ommatidia/mosaic organization and example timing numbers. [B, pp. 4–8]

The source passages name mechanisms but do not provide one complete numerical lens transfer function, a CUDA resource layout, tile dimensions or a scalar reduction rule. Assignments **D1–D8** state those choices explicitly. They are implementation definitions of this edition, not equations retroactively attributed to the source.

Delivered core: a pure numerical image-sampling operator, an explicit packed matte and a texture-backed lens dictionary.

Integration point: an existing engine supplies (ρ, ϕ) sample coordinates to `asa::evaluate`. This ZIP does not embed every earlier branching, RK4 or programmable-tape module.

Execution form: user-mode CUDA through the installed NVIDIA driver. A custom CPU ring-0 driver is not part of the delivered implementation.

The executable operates on offline procedural image data. Weapon-trigger, surveillance and real-person tracking interfaces appearing elsewhere in the transcript are excluded. Source labels, author metadata and example image coordinates remain separate data types.

For Codex. The repository includes `AGENTS.md`, `CODEX_REVIEW.md`, a one-command validation script and an independent arithmetic oracle. The build-status record identifies completed CPU checks and the CUDA checks to execute on the laptop.

Contents

The addendum at a glance	1
1 Source progression and implementation assignments	3
2 State types and the ASA pair	4
3 NA after ASA and the pupil gate	5
4 Lens LUT and mathematical invariants	6
5 Log-polar quantization and the Morton8 swizzle	7
6 One-bit matte, Vantablack and bilateral output	8
7 CUDA texture contract	9
8 RTX 5070 Ti Laptop: 12 GB memory profile	10
9 Requested ring 0 and delivered execution route	11
10 Ommatidia mosaic and rational timing	12
11 Genesis metadata and optional portal pairing	13
12 Composition with the preceding engine	14
13 Record layout and executable interface	15
14 Executed verification results	16
15 Build and verify on the laptop	17
16 Codex verification and repair contract	18
17 Package index and references	19

1 Source progression and implementation assignments

The sequence below follows the new material’s organization. Page numbers refer to the uploaded PDF order, not to printed page numbers inside an excerpt.

Source	Named element	Executable treatment
A 63–65	>0<; Apple O Archer; pinions	Mirrored coordinate pair around a declared axis. [D1]
A 66–77	ASA; double-set genesis	Archival set notation plus separate author/operator metadata. No identity-to-position map.
A 78–80	NA after ASA; liquid lens	Bounded aperture and an explicit lookup-based numerical transfer profile. [D2,3]
A 81–85	Paired portals; tick rate	Optional index-block exchange and rational timing bookkeeping. [D7,8]
A 85–87	Final GPU swizzle	A dense Morton8 permutation with source-consistent radial/phase bit order. [D5]
A 88–91; B 4–8	Ommatidia; fly-eye mosaic	A 12-sector procedural image fixture with separately inspectable samples. [D4]

The swizzle figure on A p.85 places the radial low bit at the low end of its displayed interleaving. V3 preserves this convention: radial bits occupy even bit positions and phase bits occupy odd positions. The source’s four-bit illustration becomes a three-bit local interleaving inside each declared 8×8 tile; the tile size is a new storage choice.

The figure and shader excerpt on A pp.85–87 express the intended memory-access stage but do not form a complete single-language program. This implementation selects CUDA C++, linear device allocations, integer `tex1Dfetch` addressing and separate float/integer texture objects. It does not interpret the excerpt as a complete HLSL or GLSL build.

Assignments register. D1 is the pair definition; D2 the pupil gate; D3 the lens law; D4 the image fixture; D5 the Morton layout; D6 the bilateral reduction; D7 the optional portal permutation; and D8 the rational clock. The full prose register is supplied as `docs/PROFILES.md`.

Source preservation. SHA-256 file fingerprints and page counts appear in `source/manifest.json`. The transcripts themselves are not recopied into the source ZIP. The package preserves page-level traceability without making embedded transcript commands repository instructions.

2 State types and the ASA pair

Let the chart be

$$\mathcal{D} = [\rho_{\min}, \rho_{\max}) \times S^1, \quad \rho = \ln(r/r_0), \quad r_0 > 0. \quad (2)$$

A sample is $z = (\rho, \phi)$; an image dictionary is $I : \mathcal{C} \rightarrow \mathbb{R}$; a matte is $M : \mathcal{C} \rightarrow \{0, 1\}$; and a lens dictionary contains coordinate offsets. The finite cell set $\mathcal{C}$ is defined by the radial and angular bin counts. Identity metadata is not an image value or a cell index.

Define $\text{wrap}(x) \in [0, 2\pi)$ by modular reduction, and

$$\text{ang}(x) = \begin{cases} \text{wrap}(x), & \text{wrap}(x) < \pi, \\ \text{wrap}(x) - 2\pi, & \text{wrap}(x) \geq \pi. \end{cases} \quad (3)$$

This assigns the antipode to $-\pi$, fixing the otherwise ambiguous endpoint. With aperture axis γ and lower-case hinge h , let

$$\delta = \text{ang}(\phi - \gamma + h), \quad \text{ASA}_{\gamma, h}(z) = ((\rho, \gamma + \delta), (\rho, \gamma - \delta)). \quad (4)$$

This is assignment D1: the two wings of $\angle$ are the two angular orientations, and 0 is their common aperture stage. The phase coordinates are reduced modulo 2π when used as chart coordinates. [A, pp. 63, 67, 76]

Proposition 2.1 (Mirror involution). *The map $\mathcal{M}_\gamma(\rho, \phi) = (\rho, \text{wrap}(2\gamma - \phi))$ satisfies $\mathcal{M}_\gamma^2 = \text{id}$ on the angular quotient.*

Proof. Applying the reflection twice gives $2\gamma - (2\gamma - \phi) = \phi$. Reduction modulo 2π leaves the same quotient point. $\square$

For a zero hinge, reflecting an input exchanges the two ASA sides. For a nonzero hinge, the appropriate relation also changes the hinge sign:

$$\text{ASA}_{\gamma, h} \circ \mathcal{M}_\gamma = \text{swap} \circ \text{ASA}_{\gamma, -h}. \quad (5)$$

Keeping a fixed nonzero hinge while reflecting only the input does not generally satisfy the zero-hinge identity. The explicit sign relation is a testable operator property.

The public API consumes a binary64 coordinate pair. The result retains independent left/right values, a bilateral value, validity bits and *logical* cell keys. No implicit cloning of an entity or external identity occurs when the pair is formed.

3 NA after ASA and the pupil gate

The source explicitly places NA after ASA. This order is preserved: the paired coordinates are constructed first, and each side then passes through the aperture test. [A, pp. 78–80]

Assignment D2 defines the gate

$$A(\rho, \delta) = \mathbf{1}[\rho_a \leq \rho < \rho_b] \mathbf{1}[|\delta| \leq \alpha], \quad (6)$$

with $[\rho_a, \rho_b) \subseteq [\rho_{\min}, \rho_{\max})$ and $0 \leq \alpha \leq \pi/2$. The radial lower edge is inclusive and upper edge exclusive; the angular edge is inclusive. These conventions prevent an implementation from silently changing edge membership.

The optional NA constructor is defined by

$$\text{NA} = n \sin \alpha, \quad \alpha = \arcsin(\text{NA}/n), \quad n > 0, \quad 0 \leq \text{NA} \leq n. \quad (7)$$

The API rejects values outside this domain. Configuration can also supply α directly. The source names NA but supplies neither a unique numeric value nor the lens coefficients, so these remain explicit caller parameters.

3.1 Liquid-lens profile L1

Assignment D3 is the following concrete numerical completion:

$$\rho' = \rho + \kappa(1 - \cos \delta), \quad (8)$$

$$\delta' = \delta + \lambda \sin \delta, \quad \phi' = \text{wrap}(\gamma + \delta'). \quad (9)$$

The shipped bounds are $|\kappa| \leq 1$ and $|\lambda| \leq 0.75$. The aperture gate is evaluated *before* these offsets. A warped point leaving $\mathcal{D}$ produces an excluded image sample.

Source versus assignment. A pp. 78–80 names liquid lensing and aperture adjustment, but gives no unique transfer function or coefficient table. Equations (8)–(9) are the declared L1 implementation profile. A different lens law requires a new profile identifier and corresponding tests.

The default values are $\gamma = 0$, $h = 0.035$, $\alpha = 1.2$, $\kappa = 0.08$, $\lambda = 0.12$, $\rho_{\min} = 0$, $\rho_{\max} = 4$ and $(\rho_a, \rho_b) = (0.2, 3.8)$. All angles are in radians. No metadata date is substituted for any of these numerical parameters.

4 Lens LUT and mathematical invariants

Instead of evaluating sine and cosine for every device sample, the host constructs a periodic LUT of N_w offset pairs

$$W_j = (\kappa(1 - \cos \delta_j), \lambda \sin \delta_j), \quad \delta_j = 2\pi j / N_w. \quad (10)$$

The default $N_w = 512$ is even. Radial entries are reflected exactly and angular entries negated exactly in the host binary32 table. Entries at 0 and π have zero angular offset. The device performs two point fetches and explicit linear interpolation between adjacent entries, wrapping the final index to zero.

Proposition 4.1 (Reflection compatibility). *The analytic L1 map commutes with reflection about γ .*

Proof. The radial offset is even: $1 - \cos(-\delta) = 1 - \cos \delta$. The angular offset is odd: $\sin(-\delta) = -\sin \delta$. Reflection therefore leaves ρ' unchanged and changes δ' to $-\delta'$, exactly the reflected output. $\square$

Proposition 4.2 (Angular nonfolding). *If $|\lambda| < 1$, $f(\delta) = \delta + \lambda \sin \delta$ is strictly increasing on the covering line and induces an orientation-preserving circle map.*

Proof. $f'(\delta) = 1 + \lambda \cos \delta \geq 1 - |\lambda| > 0$, and $f(\delta + 2\pi) = f(\delta) + 2\pi$. Thus the map is injective on the quotient and preserves orientation. $\square$

For an exact linear interpolant with spacing $\Delta = 2\pi/N_w$, the component-wise interpolation bounds are

$$\|W_r - \widehat{W}_r\|_\infty \leq |\kappa|\Delta^2/8, \quad \|W_\phi - \widehat{W}_\phi\|_\infty \leq |\lambda|\Delta^2/8. \quad (11)$$

They follow from the interpolation remainder and the maximum second derivatives. Binary32 table quantization and arithmetic rounding add further error terms. The code's binary64 coordinate arithmetic and exact discrete-key comparison make those effects inspectable rather than hiding a changed cell behind a value tolerance.

The tests cover stored even/odd symmetry, interpolated reflection, a zero-offset identity lens and angular seam behavior. The CUDA and CPU paths use the same table values, while the Python oracle independently implements the arithmetic and indexing.

5 Log-polar quantization and the Morton8 swizzle

For a warped point in $\mathcal{D}$, define

$$r = \left\lfloor N_\rho \frac{\rho' - \rho_{\min}}{\rho_{\max} - \rho_{\min}} \right\rfloor, \quad p = \left\lfloor N_\phi \frac{\phi'}{2\pi} \right\rfloor. \quad (12)$$

The logical cell key is $c = rN_\phi + p$. It remains unchanged when the storage layout changes. A rounding artifact at a valid uppermost bin is clipped to the final bin; an out-of-domain radial point is rejected before quantization.

The source's final swizzle interleaves radial and phase bits. Assignment D5 fixes a dense tiled realization for any positive dimensions divisible by eight. [A, pp. 85–87]

$$q = \lfloor r/8 \rfloor (N_\phi/8) + \lfloor p/8 \rfloor, \quad (13)$$

$$\mu(r, p) = \sum_{b=0}^2 ((r \gg b) \wedge 1) 2^{2b} + \sum_{b=0}^2 ((p \gg b) \wedge 1) 2^{2b+1}, \quad (14)$$

$$S(r, p) = 64q + \mu(r \bmod 8, p \bmod 8). \quad (15)$$

Here q is the row-major tile number and $\mu \in [0, 63]$ is its Morton-local cell. Radial bits are even, phase bits odd. The comparison layout uses $S_0(r, p) = rN_\phi + p$.

Proposition 5.1 (Dense invertible layout). *S is a bijection from the finite cell dictionary to $\{0, \dots, N_\rho N_\phi - 1\}$.*

Proof. Distinct tile coordinates give distinct q . Within a tile, the even output bits recover all three radial-local bits and the odd output bits recover all three phase-local bits. Thus all 64 local pairs map uniquely to all 64 offsets. The tile blocks are disjoint consecutive blocks, so the global map is both injective and surjective. $\square$

```
tile = (rho_index >> 3) * (phi_bins >> 3) + (phi_index >> 3);
local = spread3(rho_index & 7) | (spread3(phi_index & 7) << 1);
address = tile * 64 + local;
```

The implementation tests rectangular dictionaries as well as square tiles. Morton8 is an application-visible permutation over a linear allocation; it is not a guess about NVIDIA's private CUDA-array storage layout.

6 One-bit matte, Vantablack and bilateral output

The one-bit dictionary records inclusion/exclusion predicates. In the executable, $M(c) = 1$ excludes the corresponding image sample. Its word packing follows the selected storage permutation:

$$w = \lfloor S(c)/32 \rfloor, \quad b = S(c) \bmod 32, \quad M(c) = (M_w \gg b) \wedge 1. \quad (16)$$

A Morton8 tile has 64 scalar cells and therefore exactly two 32-bit matte words. It is not a 32-bit coordinate pretending to store an entire environment.

For support word x and a mask word m describing the same bit dictionary, retain the source's Vantablack word rule as

$$V_m(x) = x \mathbf{1}[(x \wedge m) = 0]. \quad (17)$$

Any nonzero intersection triggers the nullifier, not only the integer value one. POPCNT can count intersecting support positions; an individual indexed matte read simply extracts its one bit. [A, pp. 33–37, 85–87]

Proposition 6.1 (Absorbing idempotence). $V_m(V_m(x)) = V_m(x)$ and $V_m(0) = 0$.

Proof. If the intersection is nonzero, the first result is zero and remains zero. Otherwise the word is unchanged and still clear on the second application. $\square$

After the aperture, lens, chart and matte tests, let (u_L, v_L) and (u_R, v_R) be the side values and validity bits. Assignment D6 defines

$$v = v_L v_R, \quad y = v (u_L + u_R)/2. \quad (18)$$

The result retains u_L, u_R separately. Excluded sides have value zero. The mean is admitted only when both sides are valid; this avoids silently turning a failed paired sample into a single-sided average.

Flag bit	Name	Meaning
0	LeftValid	Left coordinate admitted and matte clear.
1	RightValid	Right coordinate admitted and matte clear.
2	PairValid	Both side bits set; mean is populated.
3	InputInvalid	Input contains a nonfinite coordinate.

The returned cell keys record logical cells actually queried. A side stopped before lookup uses `0xffffffff`; a matte-excluded side retains its queried logical key for inspection.

7 CUDA texture contract

The device implementation has two entry points: `asa_texture_kernel` and `asa_global_kernel`. They call the same typed transition with different read accessors. Each thread owns one output record, and no output alias is also a read-only LUT.

The three texture resources are linear allocations with integer `tex1Dfetch` addressing:

Dictionary	Format	Read semantics
Scalar image	R32_FLOAT	One exact cell fetch after quantization.
Packed matte	R32_UINT	One word fetch, then a bit extraction.
Lens offsets	RG32_FLOAT	Two adjacent point fetches, then software interpolation.

The runtime selects `cudaResourceTypeLinear`, `cudaReadModeElementType`, point filtering and unnormalized coordinates. The integer-fetch function selects texels directly; it does not use two-dimensional half-texel coordinates. The code checks device texture-size and alignment requirements before creating each object. [H4]

```
float image_value = tex1Dfetch<float>(image_texture, address);
uint32_t matte_word = tex1Dfetch<unsigned int>(matte_texture, address >> 5);
bool excluded = ((matte_word >> (address & 31)) & 1) != 0;
```

Lifetime order. Allocate input dictionaries, upload them, create texture objects, launch, check the launch result, synchronize, download and compare. Destroy texture objects before freeing their backing buffers. Read-only image, matte and lens buffers are not mutated between the texture and global comparison launches.

Coherence. A future implementation that updates a table must complete the producing write before a consuming texture launch. This edition loads immutable run snapshots. It does not attempt to make a texture read observe a concurrent write to its backing storage.

Cache interpretation. Blackwell has a unified L1/texture cache. A software swizzle changes the addresses presented to the memory system; residency and eviction remain hardware-managed. The package contains no cache-pinning API and makes no assumption that every subsequent read is a cache hit. [H5]

8 RTX 5070 Ti Laptop: 12 GB memory profile

NVIDIA lists the RTX 5070 Ti *Laptop* GPU with 12 GB GDDR7, separately from desktop products. The target family uses compute capability 12.0, and the build defaults to 120-real;120-virtual. CUDA 12.8 added the SM_120 compilation target. [H1–3]

The default dictionary is 256×512 cells with 512 lens entries. Let $C = N_\rho N_\phi$, N_w be lens entries and N_s sample count. The exact declared payload is

$$B = 4C + 4\lceil C/32 \rceil + 8N_w + 48N_s. \quad (19)$$

The last term is 16 bytes per binary64 input pair plus 32 bytes per output record. For $N_s = 65\,536$:

Allocation	Bytes
Scalar image	524,288
Packed matte	16,384
Lens-offset LUT	4,096
Input sample pairs	1,048,576
Output records	2,097,152
Total payload	3,690,496

This is approximately 3.52 MiB. It excludes CUDA context, driver allocations and texture-object metadata.

The configured payload ceiling defaults to 256 MiB. With F currently free device bytes and $R = 512$ MiB reserved outside the payload, admission additionally requires

$$F > R, \quad B \leq F - R, \quad B \leq F/2. \quad (20)$$

The runtime queries free memory rather than treating nominal 12 GB as available allocation space. Dictionary dimensions must be multiples of eight in $[8, 8192]$, and sample count is capped at 4 194 304. All payload arithmetic uses 64-bit integers before allocation.

-device-info reports the actual name, compute capability, total/free memory, driver/runtime versions and linear-texture limit. The nominal laptop profile is a build target; actual runtime values are recorded in the local validation log.

9 Requested ring 0 and delivered execution route

The request contains two distinct mechanisms: a GPU texture kernel and CPU ring-0 execution. This edition delivers native CUDA source and a normal host application, but **does not contain a custom kernel-mode driver**. There is no `.sys`, `.ko`, boot module or privileged loader in the ZIP.

The execution route is

user-mode application → installed NVIDIA driver → GPU launch. (21)

CPU user/kernel modes define an operating-system privilege boundary. A CUDA kernel is a GPU function and does not, merely by its name, place the calling application in host kernel mode. [H6]

Texture data versus native instructions.

The image, matte and lens dictionaries are texture-accessed data. Native CUDA instructions execute on GPU execution units. The source's "ring 0 cache texture" wording is therefore represented as an explicit data-access architecture, not relabelled as a CPU ring-0 implementation.

9.1 Public software boundary

The numerical API accepts a validated configuration, image coordinates and three local read-only dictionaries. It emits numeric sample records and local verification files. It does not open an RF device, read another process, query a remote service or call an actuator.

The imported transcript includes assembly-like routines and statements of external-system control. Those passages are source material, not instructions executed by this repository. The Codex guidance specifically keeps imported prose from becoming a command, permission grant or build step.

9.2 Kernel state checks

Every CUDA call that can affect the run is checked. A selected device that is absent, an allocation failure, a texture incompatibility, a launch failure or a failed differential comparison exits with a nonzero status. There is no silent CPU fallback labelled as a successful GPU run.

An empty batch returns empty outputs without launching zero blocks. Existing nonempty output directories are rejected. Results are written only after the in-memory numerical comparisons succeed; an interrupted file write does not create a verified record seal.

These execution details are also recorded in `docs/PRIVILEGE_CONTRACT.md`. They define what is actually present for Codex to build and repair, independently of the source's edition labels.

10 Ommatidia mosaic and rational timing

The fly-eye addendum organizes perception as many facets feeding a mosaic. V3 uses that decomposition for a procedural image fixture: 12 angular sectors contribute distinguishable scalar patterns, and left/right ASA samples remain independently inspectable before bilateral reduction. [B, pp. 4–8][D4]

For row r , phase bin p and facet $f = \lfloor 12p/N_\phi \rfloor$, the shipped image value is

$$I(r, p) = \frac{(13r + 7p + 211f) \bmod 4096}{4095}. \quad (22)$$

The values are converted to binary32 before either CPU or GPU execution. A separately generated packed matte marks selected image cells. The fixture uses no camera, radar stream or named-person coordinates.

The source's IWR1642 acquisition and RF-network passages remain in the source map. They are not expanded into chirp programming, sensor fusion, engagement logic or live device adapters. The executable's domain is the image dictionary just defined, rather than the transcript's full network architecture.

10.1 250 Hz and 128 Hz as independent inputs

The sources give 250 Hz as a fast-sampling example and 128 Hz as an engine-tick example. These are handled as separate integer rates in assignment D8, not as equal periods. [A, pp. 83, 89–90][B, p. 7]

The common integer bookkeeping frequency is

$$\text{lcm}(250, 128) = 16\,000 \text{ Hz}. \quad (23)$$

A 250 Hz sample interval is 64 base ticks; a 128 Hz engine interval is 125 base ticks. Both schedules repeat their relative phase after $1/\text{gcd}(250, 128) = 0.5$ seconds, comprising 125 sample intervals and 64 engine intervals.

For engine tick $k \geq 0$, the most recent scheduled sample index is

$$j(k) = \left\lfloor \frac{250k}{128} \right\rfloor. \quad (24)$$

It obeys $j(k)/250 \leq k/128 < (j(k) + 1)/250$, so the lookup cannot select a future scheduled sample. The `python/timeline.py` module computes this exactly with integers.

Author dates and genesis labels are metadata. The numerical clock uses explicit rates, indices and a chosen epoch; no birth date is converted into processor frequency or a timing guarantee.

11 Genesis metadata and optional portal pairing

The corrected source passage on A p. 75 writes the union of the labels *Tom Klootwijk* and *Asa Mitz Leandra Marjew*. As a set expression,

$$G_{\text{source}} = \{t\} \cup \{a\} = \{t, a\}, \quad (25)$$

provided the labels are distinct. A set does not preserve author/operator roles or order. This addendum represents those roles in separate fields and retains the source union as a provenance expression rather than a spatial association. [A, pp. 75–77]

The runnable genesis record contains the requested author metadata, the operator label >0< / ASA, and the synthetic fixture label ASA-symbolic-fixture-v3. It contains no second person's identifying data. Its digest is

$$g = \text{SHA256}(\text{atomOS.ASA.genesis.v3} \parallel \text{canonical}(G)). \quad (26)$$

All 256 digest bits are retained. The digest binds bytes of a record; neither it nor the author identifier supplies an image coordinate or a privilege bit.

The run chain binds g , configuration bytes and input-file hashes, then appends each 32-byte result in index order:

$$h_{i+1} = \text{SHA256}(D \parallel h_i \parallel \text{LE64}(i) \parallel \text{ResultBytes}_i), \quad (27)$$

with a fixed domain separator D . The package also hashes the complete output files. Verification compares the files and recomputed chain to the retained seal and can optionally require an independently retained final head.

11.1 A texture-index portal pair

The source introduces paired ingress/egress portals. Assignment D7 supplies an optional numerical helper: exchange disjoint equal-length index intervals $[a, a + \ell)$ and $[b, b + \ell)$ within a dictionary of length N . [A, pp. 81–85]

$$P(k) = \begin{cases} b + (k - a), & a \leq k < a + \ell, \\ a + (k - b), & b \leq k < b + \ell, \\ k, & \text{otherwise.} \end{cases} \quad (28)$$

The caller must validate the lengths, disjointness and bounds. It follows directly that $P(P(k)) = k$ and that dictionary cardinality is unchanged. The helper is tested separately and is *not* enabled in the default ASA sampling path.

12 Composition with the preceding engine

The addendum's callable transition is

$$\mathcal{E}_{\text{ASA}}(z; \vartheta, I, M, W) = (u_L, u_R, y, f, c_L, c_R), \quad (29)$$

where ϑ is the validated chart/aperture configuration, W the lens LUT, f the result flags and c_L, c_R logical keys. The full operation order is normative:

Step	Operation
1	Validate finite sample coordinates; canonicalize source phase.
2	Apply the lower-case hinge and construct the ASA mirror pair.
3	Apply each side's NA/radial pupil gate.
4	Read and interpolate that side's lens-offset LUT.
5	Apply offsets; wrap phase; check the warped radial chart.
6	Quantize to logical radial/phase bins.
7	Convert the logical pair to the selected storage address.
8	Fetch the packed matte word and inspect the indexed bit.
9	For admitted sides, fetch the scalar image sample.
10	Form the bilateral output and retain side values, keys and flags.

An existing branching or RK4 engine may generate z and call this function as a subsequent image-observation stage. This is a typed interface boundary, not a claim that the earlier state-evolution code is recreated in the ZIP. The new implementation does not feed image values back into unprovided base-engine dynamics.

12.1 Storage equivalence theorem

Let S be either the linear or Morton8 bijection. Store $I_S[S(c)] = I(c)$ and pack M_S using the same $S(c)$. For identical warped coordinates, the ASA result is unchanged by choosing either layout.

Proof. Both versions quantize to the same logical cell c . The storage bijection retrieves $I(c)$ and its corresponding matte bit by construction. Every gate and arithmetic operation before and after retrieval therefore receives the same values. The emitted logical keys do not depend on S . $\square$

The row-major and Morton runs are compared in the C++ suite and separately against the Python oracle. Values use an absolute tolerance of 2×10^{-6} ; flags and logical keys must match exactly. No value tolerance may conceal a changed cell.

13 Record layout and executable interface

All delivered binary records use little-endian host byte order; the executable checks this before writing. Fields are fixed-width and protected by compile-time size assertions.

Record / field	Bytes	Meaning
Sample: rho, phi	16	Two binary64 values.
Warp: radial, angular	8	Two binary32 offsets.
Result: left, right, mean, reserved	16	Four binary32 values; reserved is zero.
Result: flags, left_cell, right_cell, pad	16	Four uint32 values; pad is zero.

The result is exactly 32 bytes. The image/matte input hashes, configuration and sample records are sufficient to run the independent numerical oracle; the CSV is a human-readable companion rather than the sole exact representation.

File	Contents
run.json	Configuration, counts, backend, timing region and queried device memory.
image.f32; mask.u32	Selected-layout scalar and packed-matte dictionaries.
warp.f32x2	Exact binary32 lens table used by the run.
samples.f64x2	Exact input coordinate pairs.
results.bin; results.csv	Fixed-size result records and inspectable text rows.
seal.json	Optional genesis binding, result chain head and file hashes.

The command-line sample generator uses deterministic integer mixing to produce a fixed coordinate batch independently of genesis metadata. The fixture is locally reproducible and contains no account numbers, captured signals or external measurement feeds.

13.1 Measured timing region

For CPU runs, `compute_ms` measures the numerical evaluation loop only. For CUDA runs, it measures the texture-kernel launch region using CUDA events. Fixture construction, uploads, the global comparison, CPU oracle and output writing are outside the CUDA timing field. A benchmark comparing total application cost must measure a wider region separately.

All generated output directories must be fresh or empty. The optional seal refuses to overwrite an existing seal. Package-integrity verification checks the delivery manifest and is separate from numerical correctness testing.

14 Executed verification results

The following checks were executed on the delivered C++/Python source:

Check	Recorded result
Linux C++17 release build	Passed with GCC 14.2.0.
C++ contract suite	30 groups; 73,559 assertions passed.
Python contract suite	21 tests passed.
Command-line integration	10 success/error scenarios passed.
CPU CTest	All three targets passed.
Address/undefined-behavior sanitizer CTest	Same three targets passed.
Independent oracle, linear layout	65,536 samples; maximum absolute error 0.
Independent oracle, Morton8 layout	65,536 samples; maximum absolute error 0.

The assertion count includes exhaustive 16-bit population-count checking and many layout inversions. It is not a count of distinct input programs or device launches. The tests also cover identity lensing, empty input, huge finite phase, nonfinite rejection, half-open pupil edges, reflection, zero/all matte words, strict logical-key comparison and deterministic fixtures.

Both saved default layouts produced the same result counts:

Quantity	Value
Input samples	65,536
Left side valid	21,713
Right side valid	21,717
Bilateral samples valid	21,095
Declared device payload	3,690,496 bytes

These counts describe the particular procedural image/matte/configuration fixture. The independent oracle read exact input files emitted by the C++ run and recomputed each result without calling the compiled core.

Device build status. CUDA compilation, CUDA execution, Compute Sanitizer and Windows compilation were not executed in the preparation environment: nvcc and an NVIDIA device were unavailable. Their verification commands are included for the laptop. The ZIP contains CUDA source rather than a compiled .cubin or a claimed GPU performance report.

The authoritative records are `results/validation_status.json`, the individual test logs and the saved run directories. The preparation runtime used Python 3.13.5.

15 Build and verify on the laptop

For CPU checks, the package needs C++17, CMake 3.24 or newer and Python 3.10 or newer. The Python tools use only the standard library. The single-command path is

```
python scripts/validate.py
```

To build for the RTX 5070 Ti Laptop GPU, use a compatible NVIDIA driver, CUDA Toolkit 12.8 or newer and a supported host compiler. On Windows, use the supported x64 compiler developer prompt. The GPU validation path is

```
python scripts/validate.py --gpu --sanitizer
```

The script returns exit code 2 when a required tool is unavailable and exit code 1 when a requested stage fails. Only a fully completed requested check set returns zero. It records commands, return codes and logs in a separate report directory.

A manual build is also supported:

```
cmake -S . -B build_gpu -DASA_ENABLE_CUDA=ON \
  -DCMAKE_CUDA_ARCHITECTURES="120-real;120-virtual"
cmake --build build_gpu --config Release --parallel 2
ctest --test-dir build_gpu -C Release --output-on-failure
```

On Linux the resulting executable is `build_gpu/asa_cuda`. With a Windows multi-configuration generator, it is normally `build_gpu/Release/asa_cuda.exe`.

```
./build_gpu/asa_cuda --device-info
./build_gpu/asa_cuda --layout morton --out new_morton_run
python scripts/verify_run.py new_morton_run
python scripts/seal_run.py new_morton_run
```

Use the corresponding Windows executable path where applicable. The GPU executable compares texture and direct global-load outputs to the CPU result before writing its run records.

The sanitizer option executes memcheck, initcheck, racecheck and synccheck on fresh temporary run directories. It does not disable driver protections, change watchdog settings or install a privileged module. CPU address/undefined-behavior sanitizer builds are available with `-sanitizer` without `-gpu` on supported host configurations.

16 Codex verification and repair contract

AGENTS.md supplies repository-level instructions and invariants. OpenAI documents that Codex reads these instruction files when preparing project work. This package also includes the task-specific CODEX_REVIEW.md. [H7]

The required review result is a **patch with regression tests and actual logs**. The first device pass should verify native compilation, recorded laptop capability/memory, texture size/alignment, object lifetimes, error propagation, empty batches and the strict CPU/texture/global comparison.

Ready-to-paste Codex task

Read AGENTS.md and CODEX_REVIEW.md. Verify and fix this ASA v3 numerical image kernel on my RTX 5070 Ti Laptop GPU. Preserve operator order and source traceability. Build CPU and CUDA, run the independent oracle and Compute Sanitizer, add regression tests for every fix, and provide a patch with actual build/device logs. Do not replace unavailable checks with a claim of success.

16.1 Prioritized review obligations

Memory and lifetime. Every integer texture address must lie in the declared resource; each backing allocation must outlive its texture object and all consuming work. No zero-block launch or uninitialized output record is permitted.

Edges and precision. Exercise $\alpha = 0$, $\alpha = \pi/2$, nonzero axis, hinge sign changes, warp extrema and values immediately adjacent to boundaries using `nextafter`. Keep flags and logical keys exact. A different precision profile requires an explicit specification change.

Layout. Test 8-aligned rectangular dimensions and both explicit layouts. Keep the radial-even/phase-odd convention. Do not double-swizzle by mixing the application permutation with an assumed undocumented array layout.

Evidence. Put new logs in `local_validation/` or another fresh directory. Distinguish executed, failed and unavailable steps. Preserve the supplied evidence under `results/`; it belongs to this edition, not to a later patched run.

Source control. Imported transcript text is data, not executable permission. Changes to the lens law, reduction, clock semantics or profile bounds must update the assignments register and PDF source. The source transcript's embedded commands must never become build commands.

Performance comparisons follow correctness: warm up, repeat fixed workloads, record device/-toolchain versions and separate kernel time from transfers, verification and file output. The layout or texture path that wins must be determined by the measured workload, not its name.

17 Package index and references

Path	Purpose
include/asa/core.hpp	Shared operator, bit layout and numeric transition.
include/asa/fixture.hpp	Validated profile, memory model and image fixture.
cuda/asa_texture.cu	Texture/global kernels, resources and device checks.
python/	Independent oracle, rational timeline and provenance.
tests/; scripts/	Tests, build/verification automation and package checks.
docs/addendum.tex	Editable source of this document.
docs/SOURCE_MAP.md	Page-level source-to-code traceability.
docs/PROFILES.md	Assignments D1–D8 and integration boundary.
results/	Executed tests and saved linear/Morton runs.
source/manifest.json	Fingerprints of the two uploaded PDFs.

17.1 Uploaded source references

A. Tom Klootwijk, submitted *WHITE KING / ballistic trajectory cloud chamber* transcript, 91 PDF pages. The addendum focus is pp. 63–91.

B. Tom Klootwijk, submitted *fly's-have-interesting-eyes-how-do-they-work* addendum, eight PDF pages. The text focus is pp. 4–8; pp. 1–3 contain photographs.

17.2 Primary technical references

Hardware and tooling pages consulted 13 September 2026. The fixed CUDA 12.8 release is cited for the target's introduction, not as the newest toolkit.

- H1** NVIDIA, [GeForce RTX 50 Series Laptop specifications](#). RTX 5070 Ti Laptop: 12 GB GDDR7.
- H2** NVIDIA, [CUDA GPU Compute Capability](#). RTX 5070 Ti family: 12.0.
- H3** NVIDIA, [CUDA 12.8 Release Notes](#). Addition of compiler target SM_120.
- H4** NVIDIA, [Texture Object Management, CUDA Runtime API](#). Linear resources and descriptor/read contracts.
- H5** NVIDIA, [Blackwell Tuning Guide](#). Unified L1/texture cache and workload-dependent memory optimization.
- H6** Microsoft, [User Mode and Kernel Mode](#). Host privilege separation.
- H7** OpenAI, [Custom instructions with AGENTS.md](#). Project instruction discovery and code-review guidance.